

Lønssystem

Sikkerhedsrapport

Poul Schou A/S · CVR 13246505

14. september 2026

1. Baggrund

Der er gennemført en sikkerhedsgennemgang af lønssystemets kildekode med fire specialiserede analyseagenter, der har dækket følgende fire dimensioner parallelt:

Dimension	Filer gennemgået
Autentifikation og RBAC	auth.py, routers/users_router.py, database/models.py, main.py, routers/stamdata.py, routers/employees.py
Inputvalidering og injektion	routers/activities.py, routers/payroll_router.py, routers/employees.py, routers/stamdata.py, database/schemas.py
Filoperationer og path traversal	routers/import_ddd.py, routers/payroll_router.py, main.py, calculators/rates_loader.py
Frontend (XSS) og stabilitet	templates/index.html, static/js/app.js, database/session.py, calculators/overtime.py, calculators/pay_period.py

2. Implementerede rettelser

Samtlige kritiske og vigtige fund er implementeret og verificeret. Nedenstående tabel dokumenterer hvert fund, berørt fil og den anvendte rettelse.

2.1 Kritiske fund (implementeret)

Sværhedsgrad	Fund	Berørt fil	Rettelse
KRITISK	Stored XSS – 10+ steder brugte <i>innerHTML</i> med uestaped serverdata (brugernavne, kommentarer, audit log, fraværstyper). En bruger med skriveadgang kunne injicere JavaScript der kørte for alle admins.	static/js/app.js	Alle serverfelter i <i>innerHTML</i> -strengene wrappet med <i>h()</i> . <i>onclick</i> -handlers bruger nu <i>JSON.stringify()</i> . Alle fejlmeddelelser (<i>e.message</i>) escaped.

KRITISK	Path traversal via output_folder – tre endpoints accepterede en vilkårlig sti fra klienten og oprettede mapper samt skrev filer der. En payroll-bruger kunne skrive til systemkritiske stier (C:\Windows m.fl.).	routers/payroll_router.py	_safe_save_dir() tilføjet – afviser stier inden i applikationsmappen. Anvendt på proevekoersel-gem, export-csv og pdf-timesedler.
KRITISK	activity_type bypass – backend-only typer (sygdom_u_8uger, barsel_u_loen m.fl.) kunne sættes direkte via PATCH og POST, hvorved anciennitets- og lønsatslogikken blev omgået.	routers/activities.py	Guard øverst i create_manual_activity og update_activity: returnerer HTTP 400 hvis activity_type er i _BACKEND_ONLY_TYPES.

2.2 Vigtige fund (implementeret)

Sværhedsgrad	Fund	Berørt fil	Rettelse
VIGTIGT	KeyError-crash i overtidsberegning – hvis én af de tre overtidsrækker var slettet fra DB crashed hele lønkørslen for alle medarbejdere.	calculators/overtime.py	rates[KEY] → rates.get(KEY, Decimal('0')) for alle tre nøgler.
VIGTIGT	Negative sluttider accepteret – end_time ≤ start_time producerede negative timer i lønberegning og CSV-eksport.	database/schemas.py	model_validator på ActivityCreate og ActivityUpdate: kaster ValidationError hvis end_time ≤ start_time.
VIGTIGT	Hardkodet session secret som fallback – ved manglende .env startede serveren med en forudsigelig nøgle fra kildekoden, der tillod session-forfalskelse.	main.py	Fallback ændret til raise RuntimeError – serveren starter ikke uden SESSION_SECRET i .env.
VIGTIGT	Ubegrænsede filstørrelser ved .ddd-import – meget store filer blev indlæst fuldt i hukommelsen, mulig OOM.	routers/import_ddd.py	10 MB maksimal filstørrelse per .ddd-fil. Reelle filer er ~100–300 KB.
VIGTIGT	Negative satser i Stamdata – rate = -50 blev accepteret og ville producere negativ løn i alle beregninger.	routers/stamdata.py	Field(gt=0) på RateBody.rate og AgreementTypeBody.hourly_rate.

VIGTIGT	Manglende inputgrænser på skemafelter – comment og work_schedule accepterede ubegrænsede/ugyldige værdier. NB (verificeret 2026-09): trip_number (DB: String(6)) findes slet ikke i schemas.py og har derfor stadig ingen Pydantic-valideret længdegrænse – kun DB-kolonnens String(6), som SQLite ikke håndhæver.	database/schemas.py	max_length=1000 på comment, ge=0/le=24 per element i work_schedule, ge=0 på loading/unloading/km. (trip_number-grænsen er IKKE implementeret, se note.)
VIGTIGT	km_end < km_start accepteret – negativt kilometertal vises i PDF-timesedler.	database/schemas.py	model_validator: km_end ≥ km_start når begge er angivet.

3. Åbne punkter – kræver afklaring

Punkt 3.1 herunder er løst siden denne rapport blev skrevet (verificeret 2026-09). Punkt 3.2 kræver fortsat en designbeslutning om systemets rollestruktur og er ikke umiddelbart udnyttelig udefra, men bør afklares inden systemet tages i fuld drift.

3.1 Manglende rettighedsguard på medarbejder- og køretøjsendepunkter — LØST

Detalje	Beskrivelse
Status	LØST (verificeret 2026-09) – routers/employees.py og routers/vehicles.py bruger nu Depends(require_permission("manage_employees"))/("manage_vehicles")" på skriveendepunkterne. Fundet nedenfor beskriver den oprindelige tilstand.
Oprindeligt fund	Endepunkterne <i>POST /api/employees</i> (opret medarbejder), <i>PATCH /api/employees/{id}</i> (rediger medarbejder) og tilsvarende for køretøjer kræver kun at brugeren er logget ind (<i>get_current_user</i>). Der er ingen yderligere rettighedskontrol.
Konsekvens	En bruger med rollen 'disponent' – som i dag kun har rettighed til at se aktiviteter – kan teknisk set oprette, redigere og slette medarbejderstamdata og køretøjer, herunder ændre overenskomsttype og lønnummer.
Berørte filer	routers/employees.py (create_employee, update_employee) routers/vehicles.py (create_vehicle, update_vehicle, delete_vehicle)
Mulig løsning	Tilføj require_permission("stamdata") til skriveendepunkterne. Læseendepunkterne (liste, hent enkelt, overenskomsttyper) kan forblive tilgængelige for alle lognede brugere.
Afklaring krævet	Bør disponenter have mulighed for at redigere medarbejderstamdata? I dag skelner systemet ikke – alle lognede har samme skriveadgang til medarbejdere og køretøjer.

3.2 Bruger kan ændre sin egen rolle

Detalje	Beskrivelse
Fund	Endepunktet <i>PATCH /api/users/{id}</i> tillader en bruger med rettigheden 'user_management' at redigere sin egen brugerprofil, herunder at ændre sin egen rolle til en hvilken som helst anden rolle i systemet.
Konsekvens	En lønbogholder der midlertidigt er tildelt 'user_management' kan opgradere sig selv til administrator og dermed opnå fuld adgang til alle funktioner, inkl. Stamdata og lønkørsel.
Berørte filer	routers/users.py (update_user, linje ~79 og frem – linjetal drifter over tid, verificér ved tvivl) Der er allerede en guard mod at slette sin egen bruger (linje ~122), men ingen tilsvarende guard mod at ændre sin egen rolle.
Mulig løsning A	Blokér at en bruger ændrer sin egen rolle: if body.role and user.id == current_user.id: raise HTTPException(400, ...)
Mulig løsning B	Kræv at den kaldende bruger allerede har is_system=True for at tildele en is_system-rolle. Forhindrer lateral privilege escalation.
Afklaring krævet	Skal en bruger med 'user_management' overhovedet kunne tildele administrator-rollen, eller bør dette kræve at man selv er administrator?

4. Samlet vurdering

Dimension	Niveau før rettelser	Niveau efter rettelser
Autentifikation og RBAC	Acceptabelt	Stærkt ✓
Inputvalidering og injektion	Kræver arbejde	Acceptabelt ✓
Filoperationer og path traversal	Kræver arbejde	Acceptabelt ✓
Frontend (XSS)	Kræver arbejde	Stærkt ✓
Stabilitet	Acceptabelt	Stærkt ✓

Systemet er klar til stabil drift efter de implementerede rettelser. De to åbne punkter i afsnit 3 påvirker ikke systemets umiddelbare sikkerhed over for eksterne angreb, men bør afklares og implementeres inden systemet bruges af medarbejdere med forskellig tillid og adgangsniveau.